EE-2-07: Digital Design
Lab - 2 Report

Name: Dhruv Siddharth Raj Student ID: 2024UEE0117

September 26, 2025

Objective:

1. To design a VHDL(Vivado) program to create a 1-bit Ripple-carry Full Adder (FA) circuit using their *Algebraic Sum-of-Products* binary expression.
2. To design a 16-bit Ripple-carry FA using the concept of *Abstraction* and *Cascading of Logic Circuits*

Theory:

A Full Adder (FA) is a fundamental combinational circuit used to perform binary addition of three input bits: two significant bits (A and B) and a carry input (C_{in}). It produces two outputs: the Sum (S) and the Carry-out (C_{out}).
The Boolean expressions for a 1-bit Full Adder are derived using the algebraic Sum-of-Products method as follows:

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + BC_{in} + AC_{in}$$

Here is the logical circuit implementation of the above two boolean expressions. The figure is a rudimentary example of a Full Adder IC.

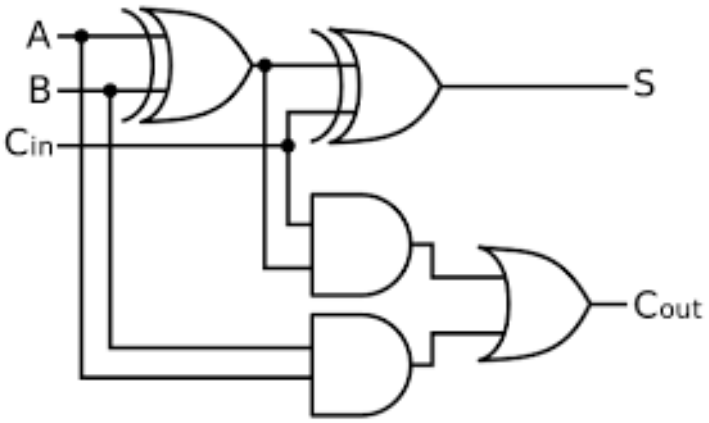

Fig.1: Logical Implementation of 1-bit Full Adder

A *Ripple-Carry Adder* is constructed by connecting multiple 1-bit Full Adders in series. The carry output from each stage is connected to the carry input of the next stage; this logically sequential connection of multiple 1-bit FAs in an iterative fashion is described as **Cascading of Logic Circuits**. It allows for the addition of multi-bit binary numbers. In a 16-bit Ripple-Carry Adder, 16 1-bit FAs are cascaded in sequence. While this design is simple, it suffers from severe time delay due to the sequential carry computation. Below is an illustration of a 4-bit Ripple Carry Adder.

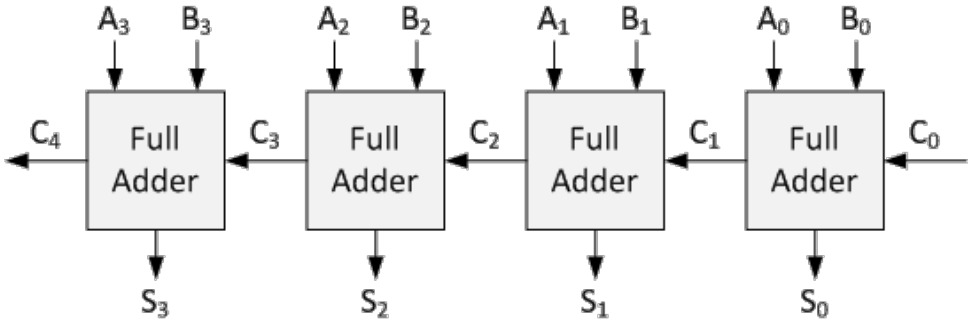

Fig.2: 4-bit Ripple Carry Adder Circuit

Here, we can take $C0$ as C_{in} and $C4$ as C_{out}, as our circuit implementation of a Full Adder IC will involve an input carry C_{in} and an output carry C_{out}.

The concept of **Abstraction** is applied by treating the 1-bit Full Adder as a modular building block or "black box." Once its design is verified, it can be replicated and interconnected without reconsidering its internal logic. Similarly, the *Cascading of Logic Circuits* principle allows the construction of higher-bit adders by chaining together multiple lower-bit adders.

Thus, by combining abstraction with cascading, a 16-bit Ripple-Carry Full Adder can be efficiently designed using the verified 1-bit Full Adder module.

Code Blocks & Output Waveforms

1-bit Full Adder VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fullAdd1bit is
    PORT(
        A : in std_logic;
        B : in std_logic;
        CIN : in std_logic;
        S : out std_logic;
        COUT : out std_logic
     );
end fullAdd1bit;
```

```
architecture Structural of fullAdd1bit is
begin
    S <= A xor B xor CIN;
    COUT <= (A and B) or (A and CIN) or (B and CIN);
end Structural;
```

Observation: Output Waveform

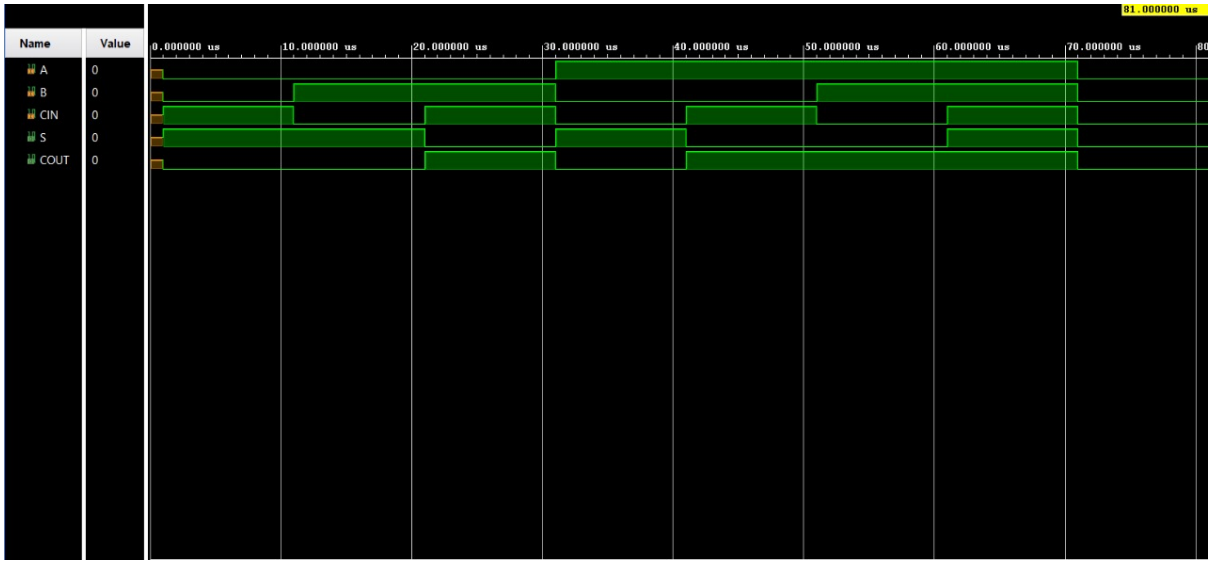

Fig.3: Output Waveform for 1-bit Full Adder Circuit

4-bit Full Adder VHDL Code

Here, we abstract the 4-bit FA circuit into 4 1-bit FA sub-circuits.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fullAdd4bit is
    PORT (
        Xin : in std_logic_vector(3 downto 0);
        Yin : in std_logic_vector(3 downto 0);
        Cin : in std_logic;
        Sout : out std_logic_vector(3 downto 0);
        Cout: out std_logic
    );
end fullAdd4bit;

architecture Structural of fullAdd4bit is
component fullAdd1bit is
    PORT(
        A : in std_logic;
        B : in std_logic;
        CIN : in std_logic;
```

```
        S : out std_logic;
        COUT : out std_logic
     );
end component;


signal c0, c1, c2 : std_logic;

begin
    S0 :  fullAdd1bit PORT MAP (
        A => Xin(0),
        B => Yin(0),
        CIN => Cin,
        S => Sout(0),
        COUT => c0
    );

    S1 :  fullAdd1bit PORT MAP (
        A => Xin(1),
        B => Yin(1),
        CIN => c0,
        S => Sout(1),
        COUT => c1
    );

    S2 :  fullAdd1bit PORT MAP (
        A => Xin(2),
        B => Yin(2),
        CIN => c1,
        S => Sout(2),
        COUT => c2
    );

    S3 :  fullAdd1bit PORT MAP (
        A => Xin(3),
        B => Yin(3),
        CIN => c2,
        S => Sout(3),
        COUT => Cout
    );

end Structural;
```

Observation: Output Waveform

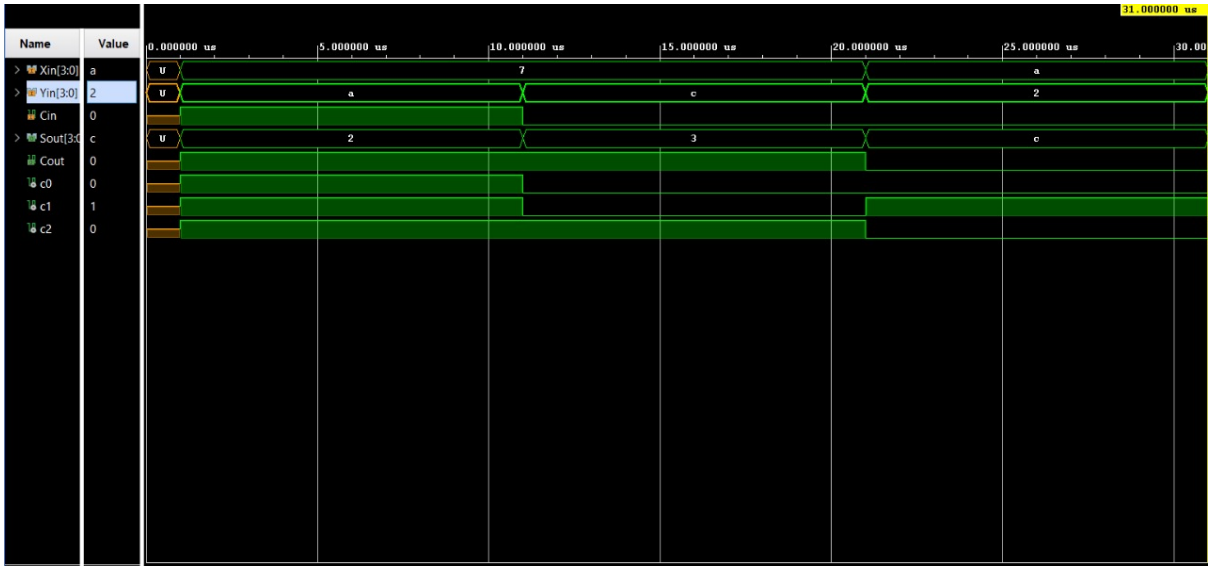

Fig.4: Output Waveform for 4-bit Full Adder Circuit

8-bit Full Adder Circuit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fullAdd8bit is
    Port ( Xin : in STD_LOGIC_VECTOR(7 downto 0);
           Yin : in STD_LOGIC_VECTOR(7 downto 0);
           Cin : in STD_LOGIC;
           Cout : out STD_LOGIC;
           S : out STD_LOGIC_VECTOR(7 downto 0));
end fullAdd8bit;

architecture Structural of fullAdd8bit is
component fullAdd4bit is
    PORT (
        Xin : in std_logic_vector(3 downto 0);
        Yin : in std_logic_vector(3 downto 0);
        Cin : in std_logic;
        Sout : out std_logic_vector(3 downto 0);
        Cout: out std_logic
    );
end component;

signal c_intermediate : std_logic;

begin
    S0 : fullAdd4bit PORT MAP (
        Xin => Xin(3 downto 0),
```

```
        Yin => Yin(3 downto 0),
        Cin => Cin,
        Sout => S(3 downto 0),
        Cout => c_intermediate
    );
    S1 : fullAdd4bit PORT MAP (
        Xin => Xin(7 downto 4),
        Yin => Yin(7 downto 4),
        Cin => c_intermediate,
        Sout => S(7 downto 4),
        Cout => Cout
    );

end Structural;
```

Observation: Output Waveform

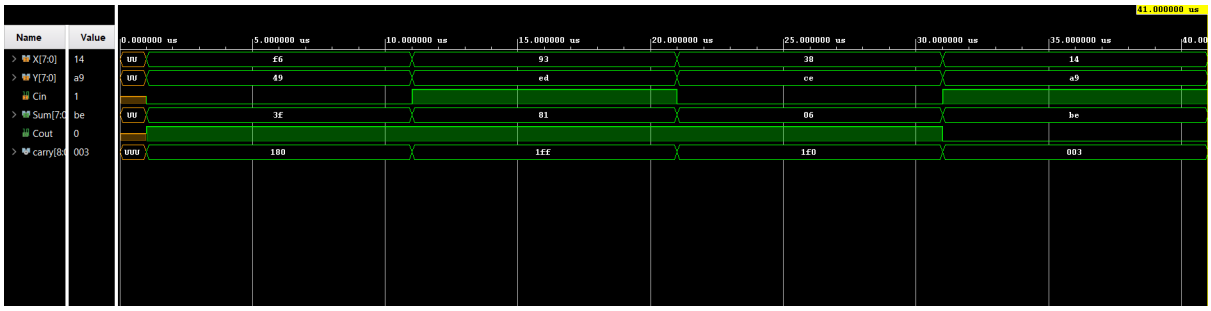

Fig.5: Output Waveform for 8-bit Full Adder Circuit

16-bit Full Adder Circuit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bit16Adder is
 PORT (
    A : in std_logic_vector(15 downto 0);
    B : in std_logic_vector(15 downto 0);
    Cin : in std_logic;
    Cout : out std_logic;
    S : out std_logic_vector(15 downto 0)
 );
end bit16Adder;

architecture Structural of bit16Adder is
    component fullAdd8bit is
        Port ( Xin : in STD_LOGIC_VECTOR (7 downto 0);
               Yin : in STD_LOGIC_VECTOR (7 downto 0);
               Cin : in STD_LOGIC;
               Cout : out STD_LOGIC;
```

```
            S : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    signal c_int : std_logic;
begin
    S0 : fullAdd8bit PORT MAP (
        Xin => A(7 downto 0),
        Yin => B(7 downto 0),
        Cin => Cin,
        S => S(7 downto 0),
        Cout => c_int
    );

    S1 : fullAdd8bit PORT MAP (
        Xin => A(15 downto 8),
        Yin => B(15 downto 8),
        Cin => c_int,
        S => S(15 downto 8),
        Cout => Cout
    );
end Structural;
```

Observation: Output Waveform

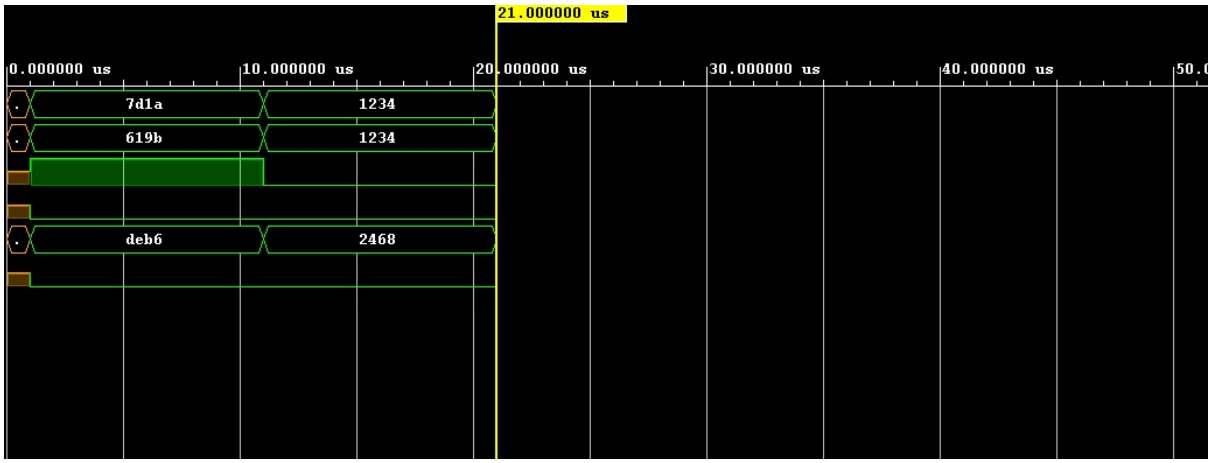

Fig.6: Output Waveform for 16-bit Full Adder Circuit

Note on Abstraction

Abstraction helps focus on essential functionality by simplifying digital circuits by hiding lower-level details. It reduces design complexity, improves understanding, and enables modularity and reuse of components.
By using abstraction, engineers can focus on what a circuit does rather than how it is implemented, making design, simulation, and troubleshooting much easier.

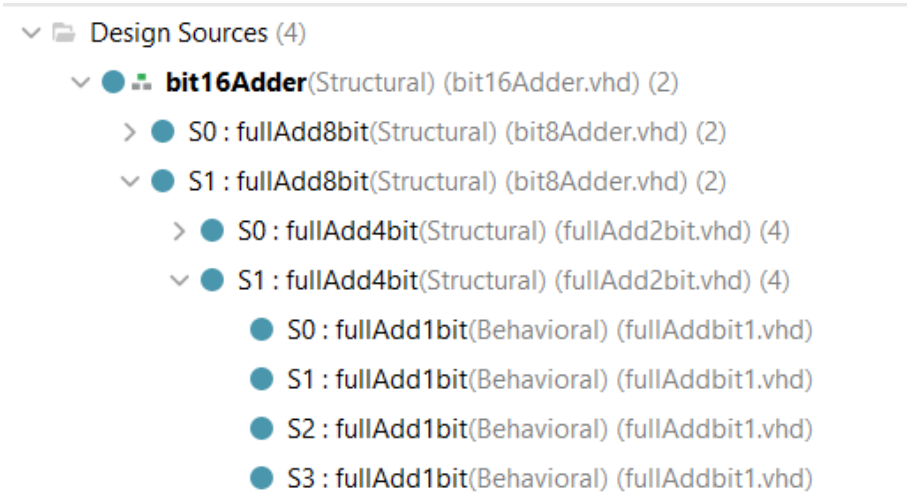

Fig.7: Observation on Circuit Abstraction

This experiment is an excellent example of how Abstraction comes in handy. Using 16 1-bit FA circuits to create 1 16-bit FA is a tedious task. However, building up towards a 16-bit FA by implementing a 4-bit FA, using it for making an 8-bit FA, and finally using an 8-bit FA to design and synthesize a 16-bit FA is much easier. Only two such similarly abstracted circuits are needed.
Thus, abstraction can be concluded as a very essential ideology which helps in understanding Gate-Level, Register-Transfer Level (RTL), and System-Level circuit characteristics and elements.

Conclusion

The 16-bit Ripple-Carry Full Adder (FA) circuit was successfully designed using the structural approach, starting with the fundamental 1-bit Full Adder, described by its Algebraic Sum-of-Products expressions. The design was extended by applying the concepts of *Abstraction* and *Cascading of Logical Circuits*. By structurally interconnecting 1-bit Full Adders, we constructed a 4-bit FA, which was further used as a building block to develop the 8-bit FA and finally a 16-bit FA.

This hierarchical design process demonstrates how complex digital systems can be constructed systematically from simpler modules, ensuring scalability, modularity, and ease of verification. This Ripple-Carry Adder design highlights both the efficiency of abstraction in digital design and the practical challenges of **propagation delay** in cascaded circuits.

Precautions

1. Ensure consistent naming of the files and variables to avoid confusion. Avoid using whitespaces in the filename and starting the filename with numbers. Also, use correct syntax to avoid unnecessary errors.

2. Use intermediate carry signals for higher order adders; i. e.; FAs that use components of smaller sub-FAs.

3. Ensure to debug all errors before starting the simulation.

4. Follow correct procedures as instructed while running simulation with hand-written inputs.

Bibliography

1. Logical Implementation of 1-bit FA: **Link**

2. Ripple Carry Adder Circuit Image: **Link**

3. Xilinx, *Vivado Design Suite*, 2024.2: **Documentation**